

UNIVERSIDAD POLITÉCNICA DE PACHUCA

Ingeniería en Tecnologías de la Información e Innovación Digital

DESARROLLO DE APLICACIONES MÓVILES

Reporte de Práctica

Implementación de Estados con remember y rememberSaveable en Jetpack Compose

Alumno: Daniel Jimenez Pluma

Grupo: TIID 04 01

Docente: Miguel Angel Montoya Cerro

Lugar: Zempoala, Hidalgo, México

Fecha: 16 de octubre de 2026

Índice general

1 Introducción **5**

1.1 Contextualización 5

1.2 Planteamiento del problema 5

1.3 Objetivo general 5

1.4 Objetivos específicos 5

1.5 Justificación 6

1.6 Estructura del documento 6

2 Marco teórico **6**

2.1 Android Studio y Kotlin 6

2.2 Jetpack Compose 6

2.3 Estado 6

2.4 Recomposición 7

2.5 mutableStateOf y remember 7

2.6 rememberSaveable 7

2.7 Modifier.clickable 7

2.8 Row, Icon y Text 7

2.9 Comparación entre variable común, remember y rememberSaveable 8

2.10 Antecedentes 8

3 Desarrollo **8**

3.1 Proyecto utilizado 8

3.2 Importaciones necesarias 9

3.3 Interfaz base 9

3.4 Contador de likes sin estado 10

3.5 Problema identificado 10

3.6 Implementación de remember 12

3.7 Pérdida del estado al rotar la pantalla 12

3.8 Implementación de rememberSaveable 13

3.9 Resultado final 14

4 Análisis y argumentación **14**

5 Resultados **15**

6 Explicación del código **15**

6.1 Importaciones de estados e interfaz 15

6.2 Actividad principal 16

6.3 Contador sin estado 16

6.4 Contador con remember 17

6.5 Contador con rememberSaveable (versión final) 17

6.6 Función app() e integración del contador 18

7 Elementos principales utilizados **19**

8 Conclusiones **20**

8.1 Aportes y limitaciones 20

8.2 Recomendaciones y proyecciones 20

9 Trabajo futuro **20**

10 Anexos **20**

10.1 Anexo A. Evidencias adicionales 20

11 Referencias **21**

Índice de ilustraciones

1 Proyecto ColumnAplication y recurso potro.png en res/drawable. 9
2 Importaciones agregadas para el manejo de estados. 9
3 Función app() con LazyColumn y el nuevo item para los likes. 10
4 Contador de likes implementado con una variable común. 10
5 El contador no se actualiza al presionar el corazón. 11
6 Contador de likes implementado con remember y mutableStateOf. 12
7 El contador se actualiza al presionar el corazón. 12
8 El contador se reinicia al rotar la pantalla con remember. 13
9 Contador de likes implementado con rememberSaveable. 13
10 El contador conserva su valor al rotar la pantalla con rememberSaveable. . 14

Índice de tablas

1 Comparación de las formas de guardar un valor 8
2 Elementos principales utilizados en la práctica 19

1. Introducción

1.1. Contextualización

El desarrollo de aplicaciones móviles requiere interfaces que respondan a las acciones del usuario. En Android, Jetpack Compose proporciona un enfoque declarativo para construir interfaces mediante funciones escritas en Kotlin: en lugar de modificar vistas de forma manual, la interfaz se describe en función de los datos que debe mostrar (Google, s. f.-a).

En este enfoque, los datos que cambian con el tiempo se denominan *estados*. Cuando un estado cambia, Compose vuelve a ejecutar las funciones que lo utilizan para actualizar la pantalla, proceso conocido como recomposición (Google, s. f.-b).

1.2. Planteamiento del problema

En Compose, las funciones composables pueden ejecutarse de nuevo cada vez que cambia un valor, por lo que una variable común declarada dentro de una función no conserva su valor entre ejecuciones y la interfaz no se actualiza. Además, cuando la actividad se destruye y se vuelve a crear, por ejemplo al rotar la pantalla, los datos que no se guardaron se pierden.

La pregunta que guía esta práctica es cómo conservar y actualizar un dato, en este caso un contador de *likes*, para que la interfaz reaccione al cambio y el valor no se pierda al rotar el dispositivo. Para resolverla se utilizaron `remember` y `rememberSaveable`.

1.3. Objetivo general

Implementar una interfaz con Jetpack Compose que utilice estados para actualizar un contador de likes, comprendiendo el funcionamiento de `remember` y `rememberSaveable` y su diferencia ante cambios de configuración.

1.4. Objetivos específicos

- Agregar a la interfaz un icono de corazón y un contador de likes.
- Utilizar `Modifier.clickable` para responder al clic del usuario.
- Identificar que una variable sin estado no actualiza la interfaz.
- Crear un estado con `remember` y `mutableStateOf`.
- Comprobar que el estado creado con `remember` se pierde al rotar la pantalla.
- Utilizar `rememberSaveable` para conservar el valor al rotar la pantalla.

1.5. Justificación

Los estados son un concepto fundamental en Jetpack Compose, ya que prácticamente cualquier interfaz interactiva depende de ellos: contadores, formularios, interruptores o listas que cambian. Comprender cómo se crean, cómo se actualizan y cuándo se pierden permite construir aplicaciones que respondan correctamente al usuario y mantengan su información ante cambios como la rotación de la pantalla.

1.6. Estructura del documento

El documento se organiza de la siguiente manera. Primero se presenta el marco teórico sobre estados, recomposición, `remember` y `rememberSaveable`. Después se describe el desarrollo de la práctica con las evidencias obtenidas, seguido del análisis, los resultados y la explicación del código por bloques. Finalmente se incluyen las conclusiones, el trabajo futuro, los anexos y las referencias.

2. Marco teórico

2.1. Android Studio y Kotlin

Android Studio es el entorno de desarrollo utilizado para crear aplicaciones Android; permite editar código, administrar recursos, compilar y probar la aplicación en emuladores o dispositivos físicos. Kotlin es el lenguaje de programación utilizado para desarrollar la aplicación y el que emplea Jetpack Compose para definir interfaces mediante funciones (JetBrains, s. f.).

2.2. Jetpack Compose

Jetpack Compose es un toolkit moderno para construir interfaces de usuario en Android mediante un enfoque declarativo. La interfaz se define con funciones anotadas con `@Composable`, en lugar de archivos XML (Google, s. f.-a).

2.3. Estado

El estado es cualquier valor que puede cambiar con el tiempo y que afecta lo que se muestra en pantalla, como el número de likes de una publicación (Google, s. f.-b). En Compose, la interfaz es una función del estado: cuando el estado cambia, las funciones composables que lo leen se vuelven a ejecutar para reflejar el nuevo valor.

2.4. Recomposición

La recomposición es el proceso mediante el cual Compose vuelve a ejecutar las funciones composables afectadas cuando cambia un estado que leen. Como una función composable puede ejecutarse muchas veces, una variable común declarada dentro de ella se reinicia en cada ejecución, por lo que no sirve para conservar datos.

2.5. mutableStateOf y remember

`mutableStateOf` crea un estado observable: cuando su valor cambia, Compose programa la recomposición de las funciones que lo leen. `remember` almacena un valor en la composición para que se conserve entre recomposiciones (Google, s. f.-b).

La combinación habitual es:

```
1 var likes by remember { mutableStateOf(0) }
2
```

La palabra `by` indica una delegación de propiedades, que permite leer y escribir el valor directamente sin usar `.value`; requiere importar `getValue` y `setValue`. El valor guardado con `remember` se conserva mientras la función composable permanezca en la composición, pero no sobrevive cuando la actividad se destruye y se vuelve a crear, como ocurre al rotar la pantalla.

2.6. rememberSaveable

`rememberSaveable` funciona como `remember`, pero además guarda el valor en el estado de instancia de la actividad, por lo que se conserva ante cambios de configuración, como la rotación de pantalla (Google, s. f.-c). Se utiliza con tipos de datos que puedan guardarse en un `Bundle`, como los números enteros.

2.7. Modifier.clickable

`Modifier.clickable` permite que un componente responda al clic o toque del usuario ejecutando el bloque de código que recibe. Es un modificador, por lo que se encadena con los demás, como `size` o `padding`.

2.8. Row, Icon y Text

`Row` organiza sus elementos de forma horizontal (Google, s. f.-d). `Icon` muestra un icono vectorial, en este caso el corazón `Icons.Filled.Favorite`, y puede teñirse con el parámetro `tint`. `Text` muestra el valor del contador.

2.9. Comparación entre variable común, remember y rememberSaveable

Tabla 1: Comparación de las formas de guardar un valor

Opción	Actualiza la interfaz	Se conserva en recomposición	Se conserva al rotar
Variable común	No	No	No
`remember` con `mutableStateOf`	Sí	Sí	No
`rememberSaveable` con `mutableStateOf`	Sí	Sí	Sí

2.10. Antecedentes

Esta práctica da continuidad a las prácticas anteriores sobre `Column`, `LazyColumn` y `LazyRow`, en las que se construyó una interfaz estática con imagen, textos y una fila horizontal. En esta práctica se reutiliza esa misma interfaz y se le agrega interacción mediante estados. Como material de apoyo se utilizó un video tutorial sobre estados en Jetpack Compose (AristiDevs, [año]).

3. Desarrollo

3.1. Proyecto utilizado

Para esta práctica se reutilizó el proyecto `ColumnAplication`, creado con la plantilla *Empty Activity* de Android Studio, que ya contiene la imagen `potro.png` en `res/drawable` y la interfaz de las prácticas anteriores.

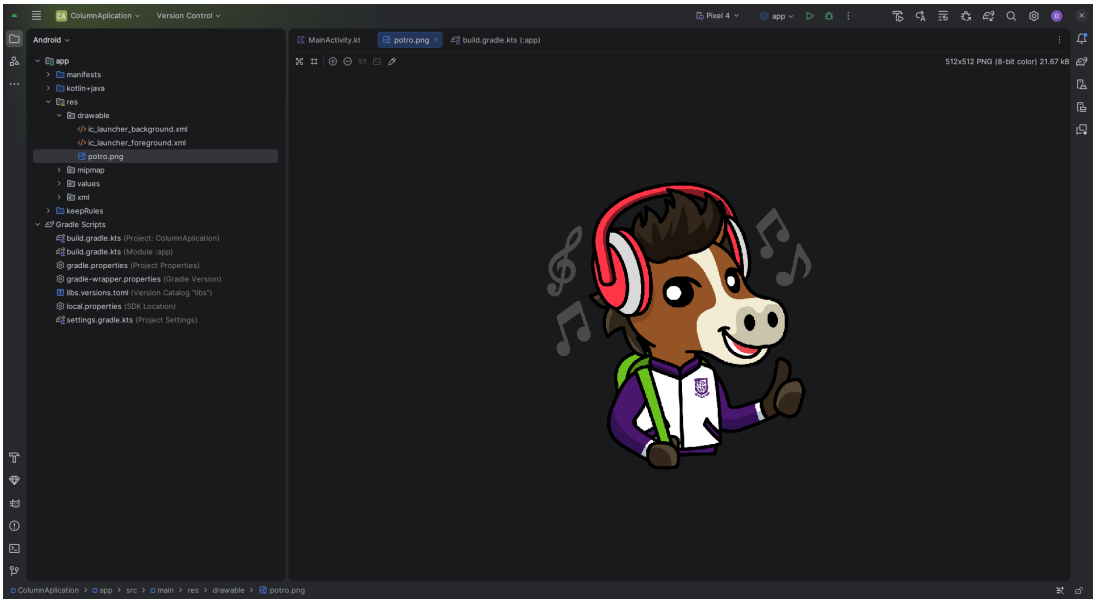


Ilustración 1: Proyecto ColumnAplication y recurso potro.png en res/drawable.

3.2. Importaciones necesarias

Se agregaron las importaciones para trabajar con estados y con el nuevo contenido de la interfaz: `Row`, `Icon`, `Icons.Filled.Favorite`, `clickable`, `size`, `Alignment`, `mutableStateOf`, `getValue`, `setValue` y `rememberSaveable`.

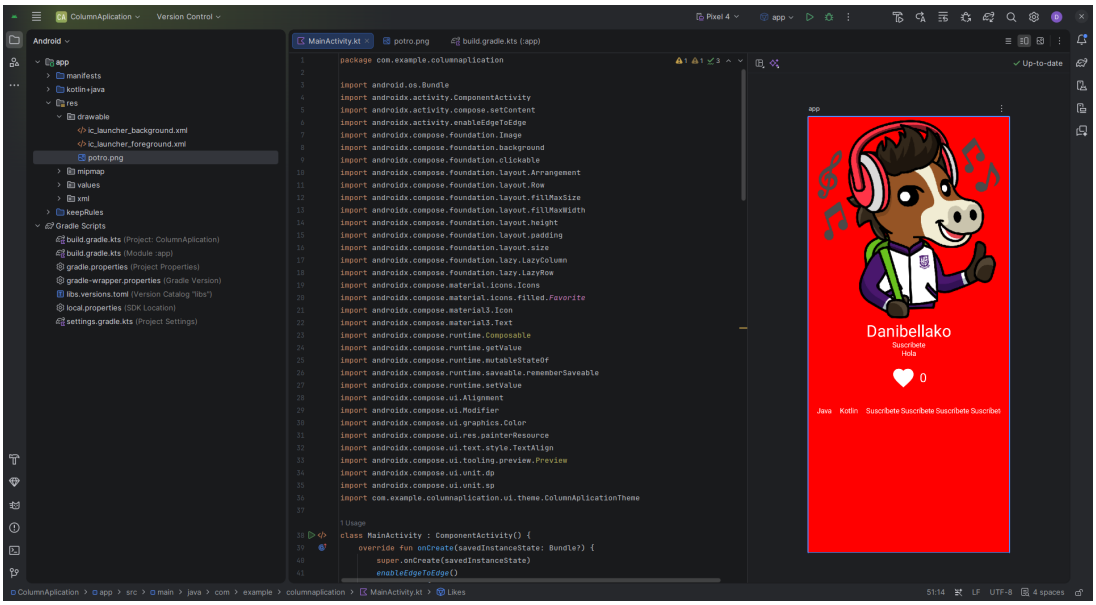


Ilustración 2: Importaciones agregadas para el manejo de estados.

3.3. Interfaz base

La función `app()` conserva la `LazyColumn` con fondo rojo, la imagen, los tres textos y la fila horizontal de las prácticas anteriores. En ella se agregó un nuevo `item` para el

contador de likes.



Ilustración 3: Función app() con LazyColumn y el nuevo item para los likes.

3.4. Contador de likes sin estado

Se creó la función composable `Likes()`, que contiene una `Row` con un icono de corazón y un texto con el contador. Al corazón se le agregó `Modifier.clickable` para incrementar el valor. En una primera versión el contador se declaró como una variable común (`var likes = 0`).



Ilustración 4: Contador de likes implementado con una variable común.

3.5. Problema identificado

Al ejecutar la aplicación y presionar el corazón se observó que el número en pantalla no cambiaba. Aunque el valor de la variable se modificaba, Compose no recibía aviso de ese cambio y por lo tanto no volvía a dibujar la interfaz. Esto permitió identificar que una variable común no es adecuada para datos que deben reflejarse en pantalla.

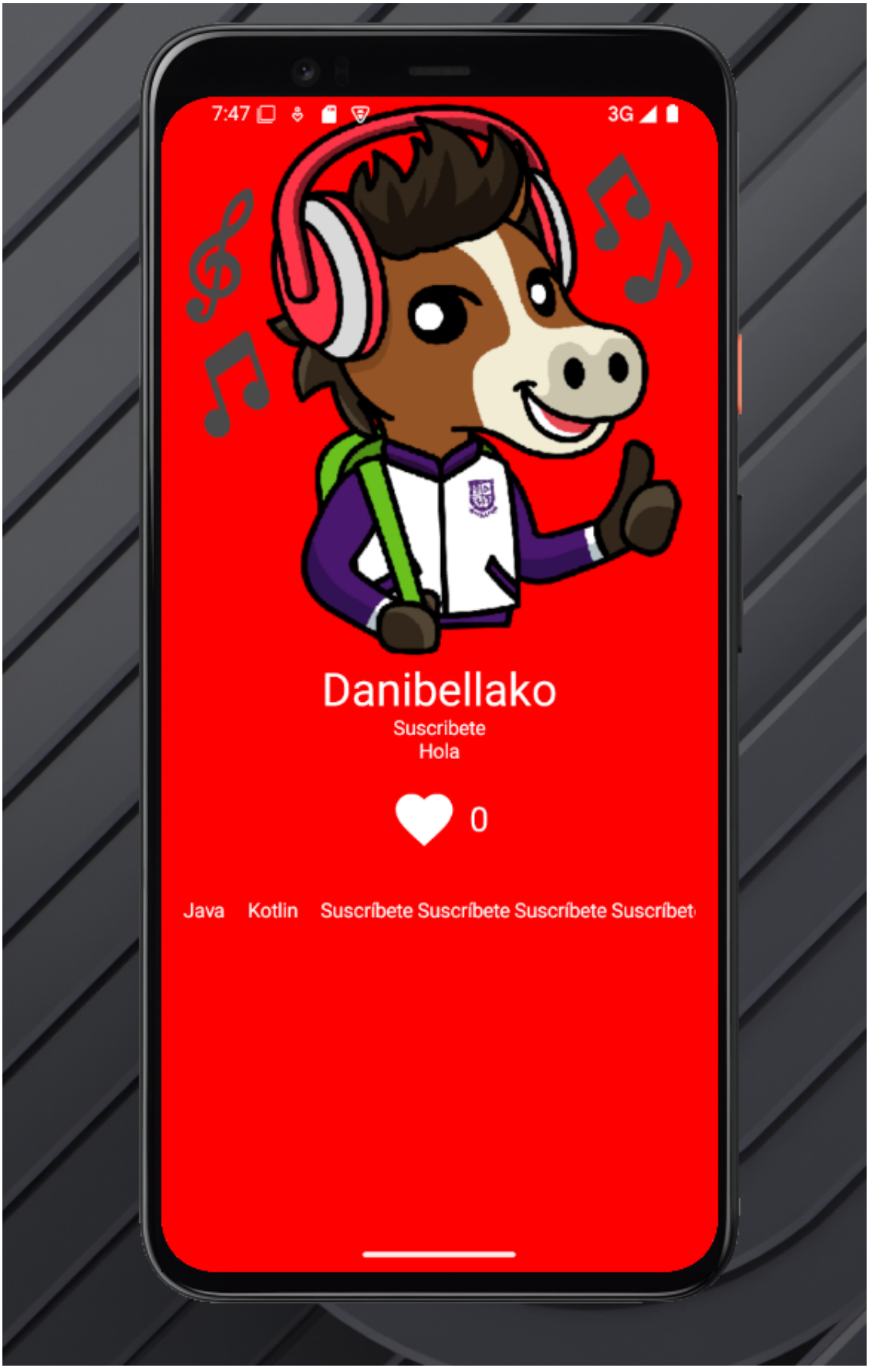


Ilustración 5: El contador no se actualiza al presionar el corazón.

3.6. Implementación de remember

Para solucionar el problema se sustituyó la variable común por un estado: `var likes by remember { mutableStateOf(0) }`. De esta forma, cada vez que cambia el valor, Compose recompone la función y el valor se conserva entre recomposiciones.

```
1 Usage
@Composable
fun Likes() {
    var likes by rememberSaveable { mutableStateOf( value = 0)}
```

Ilustración 6: Contador de likes implementado con remember y mutableStateOf.

Al ejecutar la aplicación se comprobó que el contador se incrementa en pantalla con cada clic.

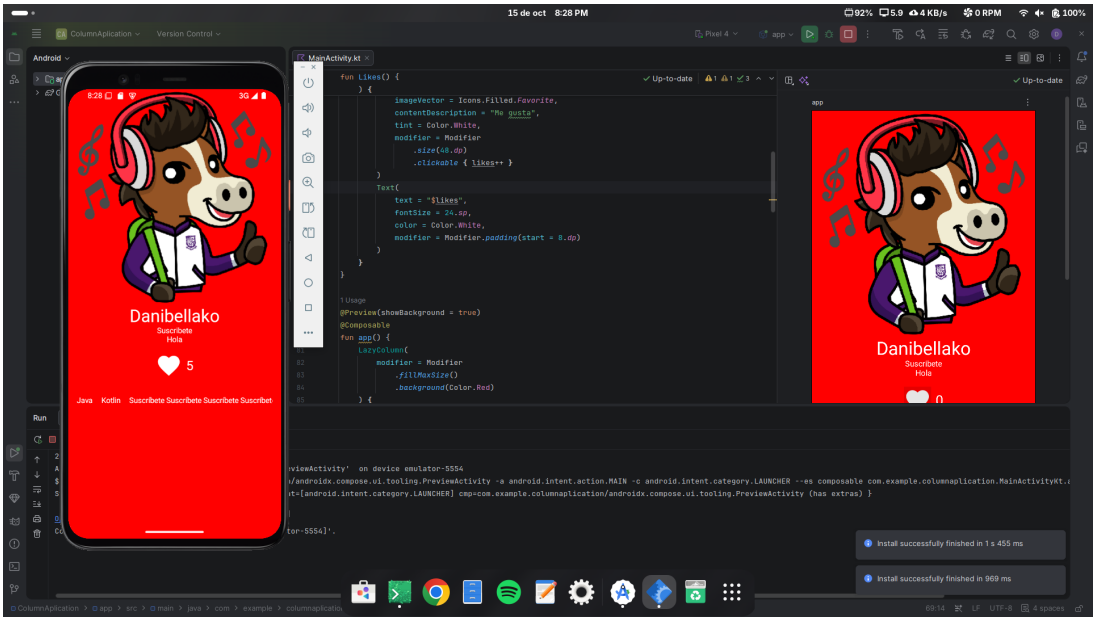


Ilustración 7: El contador se actualiza al presionar el corazón.

3.7. Pérdida del estado al rotar la pantalla

Al rotar el dispositivo con un valor distinto de cero en el contador, este volvió a mostrar cero. Al rotar la pantalla Android destruye y vuelve a crear la actividad, y el estado guardado solo con `remember` no sobrevive a ese proceso.

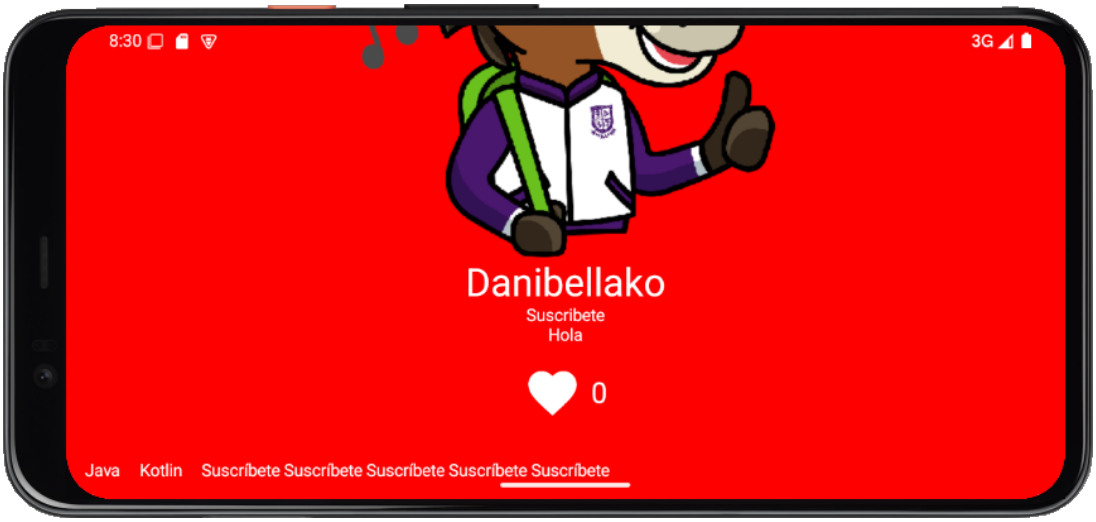


Ilustración 8: El contador se reinicia al rotar la pantalla con remember.

3.8. Implementación de rememberSaveable

Para conservar el valor al rotar la pantalla se sustituyó `remember` por `rememberSaveable`, con la declaración `var likes by rememberSaveable { mutableStateOf(0) }`.

```
1 Usage
@Composable
fun Likes() {
    var likes by rememberSaveable { mutableStateOf( value = 0)}

    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding( all = 16.dp),
        horizontalArrangement = Arrangement.Center,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            imageVector = Icons.Filled.Favorite,
            contentDescription = "Me gusta",
            tint = Color.White,
            modifier = Modifier
                .size(48.dp)
                .clickable { likes++ }
        )
        Text(
            text = "$likes",
            fontSize = 24.sp,
            color = Color.White,
            modifier = Modifier.padding(start = 8.dp)
        )
    }
}
```

Ilustración 9: Contador de likes implementado con rememberSaveable.

3.9. Resultado final

Con `rememberSaveable` el contador conserva su valor después de rotar la pantalla, por lo que la interfaz mantiene sus datos ante el cambio de configuración.

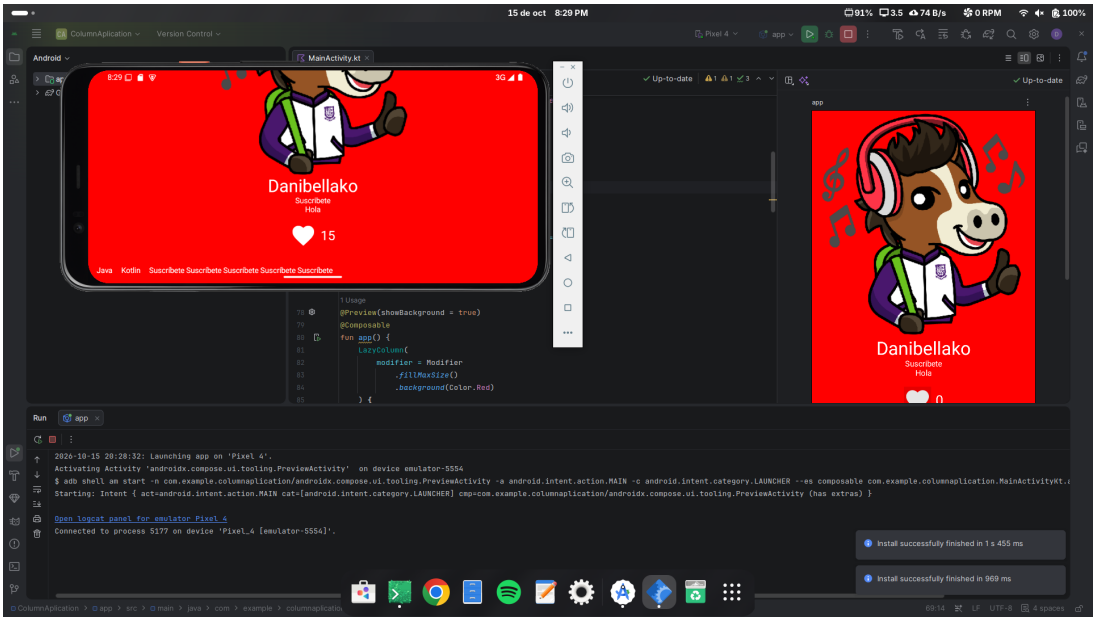


Ilustración 10: El contador conserva su valor al rotar la pantalla con rememberSaveable.

4. Análisis y argumentación

El desarrollo de la práctica permitió observar directamente el comportamiento de tres formas de guardar un valor, resumidas en la Tabla 1.

Con una variable común, el valor cambiaba internamente pero la interfaz no se actualizaba. Esto confirma lo descrito en el marco teórico: Compose solo recompone las funciones cuando cambia un estado que estas leen, y una variable común no notifica sus cambios.

Con `remember` y `mutableStateOf`, el contador se actualizó correctamente. Como `mutableStateOf` crea un estado observable, cada incremento provocó la recomposición de la función `Likes()`, y `remember` evitó que el valor se reiniciara en cada ejecución. Sin embargo, al rotar la pantalla el valor volvió a cero, porque la actividad se destruye y se vuelve a crear durante ese cambio de configuración y `remember` solo conserva el valor durante la recomposición.

Con `rememberSaveable`, el valor se conservó al rotar la pantalla, ya que esta función lo guarda en el estado de instancia de la actividad y lo restaura al volver a crearla (Google, s. f.-c). Por ello resulta la opción adecuada para datos que el usuario no debería perder, como un contador o el texto de un formulario.

También se comprobó la utilidad de `Modifier.clickable`, que permitió convertir un icono en un elemento interactivo sin necesidad de usar un botón, y de los modificadores `size` y `padding` para controlar las dimensiones y el espaciado de la fila.

La combinación de estos componentes permitió obtener una interfaz interactiva y estable ante cambios de configuración.

5. Resultados

Como resultado de la práctica se obtuvo una aplicación Android desarrollada con Kotlin y Jetpack Compose que incluye una interfaz con imagen, textos, una fila horizontal y un contador de likes interactivo.

El contador se incrementa al presionar el icono de corazón y mantiene su valor al rotar la pantalla. Se comprobó la diferencia de comportamiento entre una variable común, `remember` y `rememberSaveable`, y se cumplió el objetivo de comprender cómo se utilizan los estados para actualizar la interfaz y conservar los datos.

6. Explicación del código

A continuación se explican por bloques las partes del código desarrollado en esta práctica.

6.1. Importaciones de estados e interfaz

```
1  import androidx.compose.foundation.clickable
2  import androidx.compose.foundation.layout.Row
3  import androidx.compose.foundation.layout.size
4  import androidx.compose.material.icons.Icons
5  import androidx.compose.material.icons.filled.Favorite
6  import androidx.compose.material3.Icon
7  import androidx.compose.runtime.getValue
8  import androidx.compose.runtime.mutableStateOf
9  import androidx.compose.runtime.saveable.rememberSaveable
10 import androidx.compose.runtime.setValue
11 import androidx.compose.ui.Alignment
12
```

Descripción. Estas son las importaciones nuevas respecto a la práctica anterior. `clickable` permite responder al clic; `Row`, `size` y `Alignment` organizan y dimensionan el contenido; `Icons`, `Favorite` e `Icon` muestran el corazón; y `mutableStateOf`, `rememberSaveable`, `getValue` y `setValue` permiten crear el estado y usarlo con la palabra

by.

6.2. Actividad principal

```
class MainActivity : ComponentActivity() {
        override fun onCreate(savedInstanceState: Bundle?) {
                super.onCreate(savedInstanceState)
                enableEdgeToEdge()
                setContent {
                        ColumnAplicationTheme {
                                app()
                        }
                }
        }
}

```

Descripción. `MainActivity` es la pantalla que Android abre al iniciar la aplicación. `enableEdgeToEdge()` permite dibujar la interfaz debajo de las barras del sistema y `setContent` establece la interfaz mediante Compose, envuelta en el tema del proyecto. Al rotar la pantalla, esta actividad se destruye y se vuelve a crear.

6.3. Contador sin estado

```
@Composable
fun Likes() {
        var likes = 0

        Row(
        modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp),
        horizontalArrangement = Arrangement.Center,
        verticalAlignment = Alignment.CenterVertically
        ) {
                Icon(
                imageVector = Icons.Filled.Favorite,
                contentDescription = "Me gusta",
                tint = Color.White,
                modifier = Modifier
                .size(48.dp)
```

```
                .clickable { likes++ }
                )
                Text(
                text = "$likes",
                fontSize = 24.sp,
                color = Color.White,
                modifier = Modifier.padding(start = 8.dp)
                )
        }
}

```

Descripción. Primera versión del contador. `likes` es una variable común, por lo que aunque `likes++` modifica su valor al hacer clic, Compose no se entera del cambio y el texto en pantalla no se actualiza. La `Row` coloca el corazón y el número uno al lado del otro: `horizontalArrangement = Arrangement.Center` los centra horizontalmente y `verticalAlignment = Alignment.CenterVertically` los alinea verticalmente. El corazón es un `Icon` de 48 dp, teñido de blanco, con `Modifier.clickable` para detectar el clic.

6.4. Contador con remember

```
        var likes by remember { mutableStateOf(0) }

```

Descripción. Se sustituyó la variable común por un estado. `mutableStateOf(0)` crea un estado observable con valor inicial cero y `remember` lo conserva entre recomposiciones. Con ello, cada `likes++` provoca que Compose vuelva a ejecutar la función y muestre el nuevo valor. El resto del código no cambia. Esta versión requiere además importar `remember`.

6.5. Contador con rememberSaveable (versión final)

```
@Composable
fun Likes() {
        var likes by rememberSaveable { mutableStateOf(0) }

        Row(
        modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp),
```

```
                horizontalArrangement = Arrangement.Center,
                verticalAlignment = Alignment.CenterVertically
                ) {
                        Icon(
                        imageVector = Icons.Filled.Favorite,
                        contentDescription = "Me gusta",
                        tint = Color.White,
                        modifier = Modifier
                        .size(48.dp)
                        .clickable { likes++ }
                        )
                        Text(
                        text = "$likes",
                        fontSize = 24.sp,
                        color = Color.White,
                        modifier = Modifier.padding(start = 8.dp)
                        )
                }
        }

```

Descripción. La versión final solo cambia `remember` por `rememberSaveable`. Con ello el valor del contador se guarda en el estado de instancia de la actividad y se restaura cuando esta se vuelve a crear, por ejemplo al rotar la pantalla.

6.6. Función app() e integración del contador

```
        @Preview(showBackground = true)
        @Composable
        fun app() {
                LazyColumn(
                modifier = Modifier
                .fillMaxSize()
                .background(Color.Red)
                ) {
                        // imagen y textos de las practicas anteriores

                        item {
                                Likes()
                        }

                        // fila horizontal (LazyRow)
```

```
16                          }
17                  }
18
```

Descripción. La `LazyColumn` conserva la imagen, los textos y la fila horizontal de las prácticas anteriores. El contador se agrega en un nuevo `item` que simplemente llama a la función `Likes()`, lo que mantiene el código ordenado y permite reutilizar el componente.

7. Elementos principales utilizados

Tabla 2: Elementos principales utilizados en la práctica

Elemento	Función
`MainActivity`	Actividad principal de la aplicación.
`setContent`	Establece el contenido de la interfaz mediante Compose.
`@Composable`	Identifica una función que construye elementos de interfaz.
`@Preview`	Permite visualizar una vista previa de la interfaz.
`LazyColumn`	Organiza elementos verticalmente y permite desplazamiento.
`Row`	Organiza elementos horizontalmente.
`Icon`	Muestra un icono vectorial.
`Icons.Filled.Favorite`	Icono de corazón utilizado para los likes.
`Text`	Muestra texto.
`Modifier.clickable`	Hace que un componente responda al clic.
`mutableStateOf`	Crea un estado observable.
`remember`	Conserva un valor entre recomposiciones.
`rememberSaveable`	Conserva un valor entre recomposiciones y ante cambios de configuración.
`by`	Delegación de propiedades para leer y escribir el estado sin usar `.value`.
`Arrangement` y `Alignment`	Controlan la distribución y la alineación de los elementos.

8. Conclusiones

8.1. Aportes y limitaciones

La realización de esta práctica permitió comprender el papel de los estados en Jetpack Compose: una variable común no actualiza la interfaz, `remember` con `mutableStateOf` permite que la pantalla reaccione a los cambios, y `rememberSaveable` además conserva el valor cuando la actividad se destruye y se vuelve a crear. También se consolidó el uso de `Modifier.clickable` para crear elementos interactivos.

Entre las limitaciones se encuentra que el estado se maneja directamente dentro de la función composable, lo que es suficiente para un contador simple pero no para aplicaciones más grandes. Además, el contador solo permite sumar likes, sin quitarlos, y las pruebas se realizaron únicamente rotando la pantalla.

8.2. Recomendaciones y proyecciones

Se recomienda estudiar el patrón de elevación de estado (*state hoisting*) para separar el estado de la interfaz, y el uso de `ViewModel` para aplicaciones con más datos. También se recomienda cambiar el color del corazón según el estado para dar retroalimentación visual al usuario.

9. Trabajo futuro

Como continuación de esta práctica se pueden implementar las siguientes mejoras:

- Alternar el corazón entre me gusta y no me gusta, cambiando su color.
- Aplicar elevación de estado para reutilizar el componente `Likes`.
- Utilizar un `ViewModel` para manejar el estado de la pantalla.
- Agregar otros estados, como campos de texto o interruptores.
- Probar el comportamiento del estado en diferentes cambios de configuración.

10. Anexos

10.1. Anexo A. Evidencias adicionales

Las diez ilustraciones incluidas en el desarrollo del documento representan las principales etapas realizadas durante la práctica. Las imágenes utilizadas corresponden a los archivos `1.png` a `10.png`.

11. Referencias

- AristiDevs. ([2021]). *[Estados (STATES) en JETPACK COMPOSE - Tutorial Android JETPACK COMPOSE desde cero [2021]]* [Video]. YouTube. [enlacedelvideo]
- Google. (s. f.-a). *Jetpack Compose*. Android Developers. https://developer.android.com/compose
- Google. (s. f.-b). *Estado y Jetpack Compose*. Android Developers. https://developer.android.com/develop/ui/compose/state
- Google. (s. f.-c). *Guardar el estado de la IU en Compose*. Android Developers. https://developer.android.com/develop/ui/compose/state-saving
- Google. (s. f.-d). *Fundamentos de los diseños de Compose*. Android Developers. https://developer.android.com/develop/ui/compose/layouts/basics
- JetBrains. (s. f.). *Kotlin documentation*. Kotlin. https://kotlinlang.org/docs/home.html
- Jiménez Pluma, D. (2026). *Desarrollo de Aplicaciones Móviles* [Repositorio de código]. GitHub. https://github.com/danieljiplu-007/Desarrollo-de-Aplicaciones-Movi
- Universidad Politécnica de Pachuca. (s. f.). *Guía de estructura y desarrollo de documentos académicos*. Material proporcionado para la asignatura.